

Thursday Code Lab: Getting Started with MEEP

Prof. Jordan C. Hanson

November 20, 2025

1 The Conda Environment Manager

After installing MEEP using the Conda environment manager, we can activate our Python3 environment `mp` by typing the following in the terminal:

```
conda activate mp
```

the terminal should switch from `(base)` at the far left, to `(mp)`.

2 Near-to-Far Field Projection

In many CEM applications, we want to characterize the flow of electromagnetic energy from radiating sources in the *far-field*, many wavelengths from our source. To compute the propagation from our source to the far-field, however, is computationally expensive. To make this process more efficient, we can use a form of the multipole expansion to *project* our results in the near-field (a few wavelengths) to the far-field. This is known as near-to-far-field projection. To get started copy the following code into a Python3 script called `neartofar.py`:

```
import meep as mp
import numpy as np
import matplotlib.pyplot as plt
resolution = 5
f = 0.2
lamb = 1/f
source_length = 2
source_width = 0.2
dpml = 1.0
npts = 360
radius = 1000
angles = 2*np.pi/npts*np.arange(npts)
sources = [
    mp.Source(
        src=mp.ContinuousSource(frequency=f),
        center=mp.Vector3(0,0),
        size=mp.Vector3(source_width,source_length),
        component=mp.Y
    )
]
geometry = []
sim = mp.Simulation(
    resolution=resolution,
    cell_size=mp.Vector3(100,100),
    boundary_layers=[mp.PML(dpml)],
    sources=sources,
    geometry=geometry)
```

This code will initialize the `sim` object, along with some variables. These include frequency, wavelength, source sizes, and a radius to which we will project our radiation. Before we run the computation, we must draw a Gaussian surface around our radiation source. The following code sets up our surface, and creates a graph of the surface and the source within it.

```
d = 5.0*source_length
top = mp.Near2FarRegion(
    center=mp.Vector3(0,d/2),
    size=mp.Vector3(d,0),
    weight=+1)
bottom = mp.Near2FarRegion(
    center=mp.Vector3(0,-d/2),
    size=mp.Vector3(d,0),
    weight=-1)
right = mp.Near2FarRegion(
    center=mp.Vector3(d/2,0),
    size=mp.Vector3(0,d),
    weight=+1)
left = mp.Near2FarRegion(
    center=mp.Vector3(-d/2,0),
    size=mp.Vector3(0,d),
    weight=-1)
proj_box = sim.add_near2far(f,0,1,top,bottom,left,right)
fig = plt.figure(dpi=300)
sim.plot2D(ax=fig.gca())
plt.savefig("surfaces.png")
plt.close()
```

3 Running the CEM Simulation

Now it is time to run the simulation:

```
sim.run(until=100)
```

After the run completes, we must extract the flux data from our Gaussian surface and project it into the far-field. The following code extracts the radiation pattern in decibels (dB) versus azimuthal angle (degrees), and creates a polar graph of it:

```
E = np.zeros((npts,3),dtype=np.complex128)
H = np.zeros((npts,3),dtype=np.complex128)
for n in range(npts):
    ff = sim.get_farfield(
        proj_box,
        mp.Vector3(
            radius*np.cos(angles[n]),
            radius*np.sin(angles[n]))
    )
    E[n,:] = [ff[j] for j in range(3)]
    H[n,:] = [ff[j+3] for j in range(3)]
Px = np.real(np.conj(E[:,1])*H[:,2]-np.conj(E[:,2])*H[:,1])
Py = np.real(np.conj(E[:,2])*H[:,0]-np.conj(E[:,0])*H[:,2])
Pr = np.sqrt(np.square(Px) + np.square(Py))
directivity = 10.0*np.log10(Pr/max(Pr))
g = plt.figure(dpi=300)
plt.polar(angles,directivity)
ax = g.gca()
ax.set_rlim(-30,0)
ax.set_rticks([-15,0])
plt.show()
```